

The Fundamentals of Programming

While Loops



Objectives

What you will learn:

1. How to repeat actions using loops.
2. The different ways that loops can be used to solve complex problems.
3. To diagnose and fix problems with loops.

Why Loops?

Motivation

Programs use loops to repeat actions. For example, say we've made a function called **check_if_adult** which asks people their age and then returns True if they're an adult. Using a loop we could **repeatedly** call the function without having to do something like:

```
check_if_age()  
check_if_age()  
check_if_age()  
....
```

```
def check_if_adult():  
    age = int(input('age: '))  
    if age >= 18:  
        return True  
    return False
```

Python Loops

Background

Python actually has two different types of loops, the **for loop** and the **while loop**. The **while loop** is used when we're not sure when a situation will end.

How does this apply to the example program on the right?

```
age = 10
while age < 20:
    age = int(input("Enter age: "))

print(age)
```

The While Loop

We have already seen how to define a function and use if statements. What similarity do you notice between them and while loops?

Basic while loop:

while **conditional is True**:
do stuff here

Example:

```
x = 5
while x >= 0:
    print(x)
    x -= 1
print("Blast off")
```

While loops

Practice – Easier

With each of the following programs, state the output.

```
x = 0
while x < 10:
    x += 2
    print(x)
print(x)
```

```
x = 1
while x % 2 != 0:
    x += 2.2
    print(x)
```

```
x = 16
while x // 2 > 0:
    x = x // 2
    print(x)
```

While loops

Practice Harder

With each of the following programs, state the output.

```
a = 0
c = 10
while a < c:
    a = a + 1
    c = int(c / 2)
    print(a + c)
```

```
var1 = 0
var2 = 12
while var1 != var2:
    var1 += 2
    var2 += 2
    if var2 % 16 == 0:
        var2 -= 4
    print(var1, var2)
```

Practical

Python turtle

Turtle is a simple way to draw things with Python. It can also help illustrate some interesting thing that can be done with loops, for instance, a spiral of sorts.

To run this, use **repl.it** and create a Turtle project.

```
import turtle  
t = turtle.Turtle()
```

```
length = 4  
while length < 200:  
    turtle.forward(length)  
    turtle.left(90)  
    length += 4
```


Practical

Collatz part 1

The Collatz conjecture says that if you follow the rules below for any number that in time you will reach 1.

1. **If the number is even divide by 2.**
2. **If the number is odd, multiply by 3 and add 1.**

Complete the function to the right using a while loop which will count the number of loops needed to reach 1.

```
def collatz_count(num):  
    count = 0  
    while  
        if  
  
        else  
  
    return count  
  
print(collatz_count(10))
```

Practical

Collatz part 2

Using the function you created, loop starting from 2 and go up to and including 100. The basic structure is shown to the right.

Optional

Can you modify the program to tell you which number takes the longest??

```
def collatz_count(num):  
    ....
```

```
count = 2  
while ...
```

Recap

We learned

1. Why we need loops.
2. That the way we write while loops is similar in structure to functions and if statements in that they use indentation and colons.
3. You can use loops in combination with both if statements and functions to solve different problems.